| Threads | A Time | B Time | A Forward | B Forward | A Reverse | B Reverse |
|---|---|---|---|---|---|---|
| 1 | .2327s | .2273s | 1.00E+19 | 1.00E+19 | 1.00E+19 | 1.00E+19 |
| 2 | .9258s | .2351s | 0 | 0 | 0 | 0 |
| 4 | 2.1061s | .2247s | 199.9999997 | 199.9999997 | 0 | 0 |
| 8 | 4.2351ss | .2267s | 199.9999997 | 199.9999997 | 0 | 0 |
| 12 | 4.1675s | .2349s | 199.9999997 | 199.9999997 | 0 | 0 |

I ran the program with 1, 2, 4, 8, and 12 threads and got these results. The first thing I noticed was how different the times were between A and B once I started adding threads. With one thread they were basically the same. Run A got slower every time I added more threads, while Run B stayed around 0.23 seconds the whole time. Run A was over 4 seconds with 8 threads, while Run B was still basically the same as it was with one thread.

Looking at the code, the main difference is how the sums are stored. In A, the sums are all next to each other in memory. In B, the Separated struct puts space between them. This seems to be the main reason for the difference in performance. With A, the threads are constantly working with values that are on the same cache line, so they end up causing a lot more cache activity. B avoids most of that by separating the values.

The results for the sums were also weird. With 4, 8, and 12 threads, the forward sum was about 200, but the reverse sum was 0. The threads are all doing *data->sum += value at the same time, so there isn't anything making those updates happen safely. volatile doesn't solve that problem. Threads can basically overwrite each other's updates, which explains why the results aren't what I would expect.

What I found most interesting is that adding threads actually made A worse instead of better. Normally I would expect more threads to help, but in this case the way the memory is being accessed becomes the bottleneck. B does a much better job of avoiding that problem.

If I were changing the program, I would probably have each thread keep its sum in a local variable instead of constantly writing to shared memory. Once it finished, it could put its final result into the array. Then the main thread could combine everything after the threads finish. That would avoid a lot of the problems that are happening in the current version.